

第 9 章 机器人导航(实体)

第7章内容介绍了在仿真环境下的机器人导航，第8章内容介绍了两轮差速机器人的软硬件实现，如果你已经按照第8章内容构建了自己的机器人平台，那么就可以尝试将仿真环境下的导航功能迁移到机器人实体了，本章就主要介绍该迁移过程的实现，学习内容如下：

- 比较仿真环境与真实环境的导航实现；
- 介绍VScode远程开发的实现；
- 实体机器人导航实现。

预期达成学习目标：

- 了解仿真环境与真实环境下导航实现的区别；
- 能够搭建VScode远程开发环境；
- 能够实现实体机器人的建图与导航。

9.1 概述

实体机器人导航与仿真环境下的导航核心实现基本一致，主要区别在于导航实现之前，基本环境的搭建有所不同，比如:导航场景、传感器、机器人模型等，大致区别如下：

	仿真环境	实体机器人
导航场景	依赖于gazebo搭建的仿真环境	依赖于现实环境
传感器	在gazebo中通过插件来模拟一些列传感器，比如:雷达、摄像头、编码器....	使用的是真实的传感器
机器人模型	依赖于机器人模型，以实现仿真环境下的机器人的显示，通过robot_state_publisher、joint_state_publisher实现机器人各部件的坐标变换。	机器人模型不是必须的，如果不使用机器人可以通过static_transform_publisher发布导航必须的坐标变换；如果使用机器人模型也可以借助robot_state_publisher、joint_state_publisher 发布坐标变换，当然后者可以在rviz中显示机器人模型，更友好。

总体而言，在实体机器人导航中，可以完全脱离 gazebo，而机器人模型是否使用，可以按需选择。

除此之外，实体机器人导航的开发环境也发生了改变，程序最终需要被部署在机器人端，我们可以在本地开发，然后通过ssh上传至机器人，或者也可以通过VScode的远程开发插件直接在机器人端编写程序。

9.2 VScode远程开发

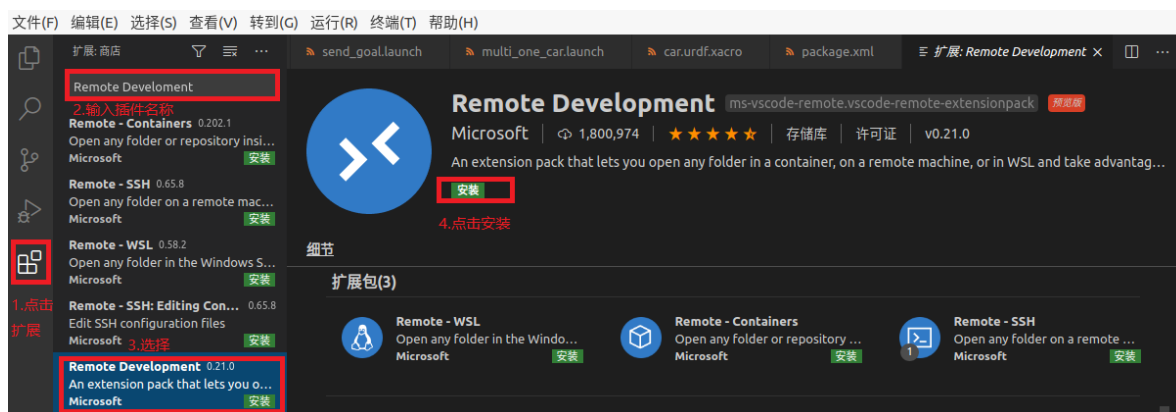
在第8章，我们介绍了ssh远程连接的使用，借助于ssh可以远程操作树莓派端，但是也存在诸多不便，比如:编辑文件内容时，需要使用vi编辑器，且在一个终端内，无法同时编辑多个文件，本节将介绍一较为实用的功能——VSCode远程开发，我们可以在VSCode中以图形化的方式在树莓派上远程开发程序，比ssh的使用更方便快捷，可以大大提高程序开发效率。

1.准备工作

VScode远程开发依赖于ssh，请首先按照8.5.3内容配置ssh远程连接。

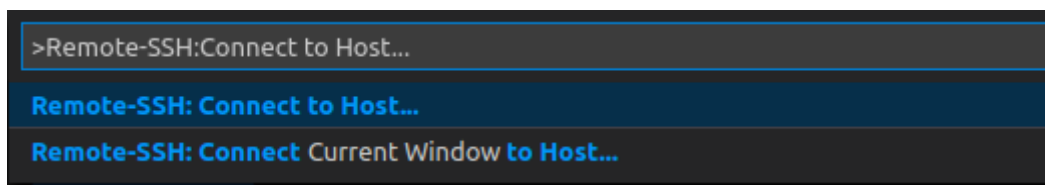
2.为VScode安装远程开发插件

启动VSCode，首先点击侧边栏的扩展按钮，然后在 **扩展：商店** 的搜索栏输入 **Remote Development** 并点击同名插件，最后在右侧显示区中点击 **安装**。

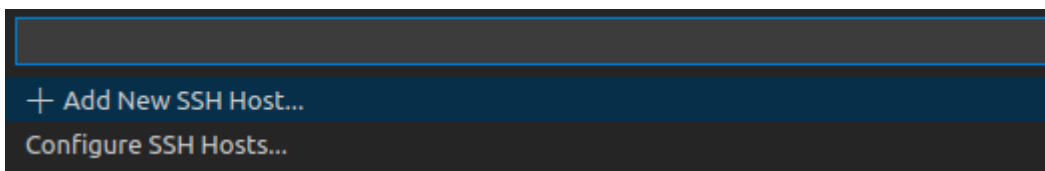


3.配置远程连接

步骤1：使用快捷键 **ctrl + shift + p** 打开命令输入窗口，并输入 **Remote-SSH:Connect to Host...**，弹出列表中选择与之同名的选项。



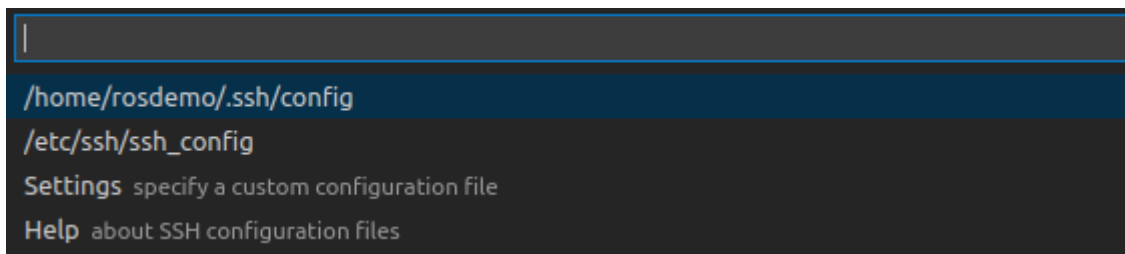
步骤2：步骤1完成将弹出一个新的命令窗口如下，选择下拉列表中的 **Add New SSH Host**。



步骤3：步骤2完成又将弹出一个新的命令窗口，在其中输入：**ssh ubuntu@192.168.43.164**，其中，**ubuntu** 需要替换为你的登录账号，**192.18.43.164** 则替换为你的树莓派的ip地址。

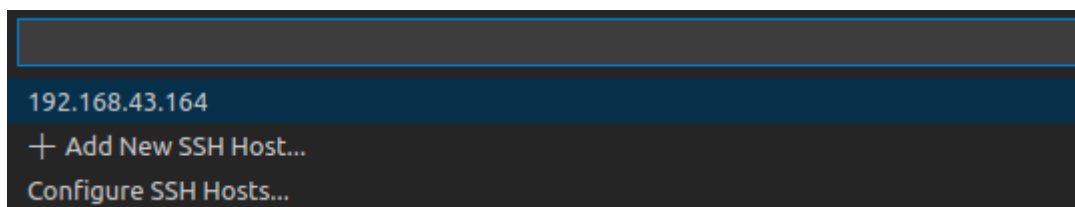


步骤4：选择步骤3完成后的弹窗列表中的第一个选项(或直接回车)，即可完成配置，配置成果后会有提示信息。

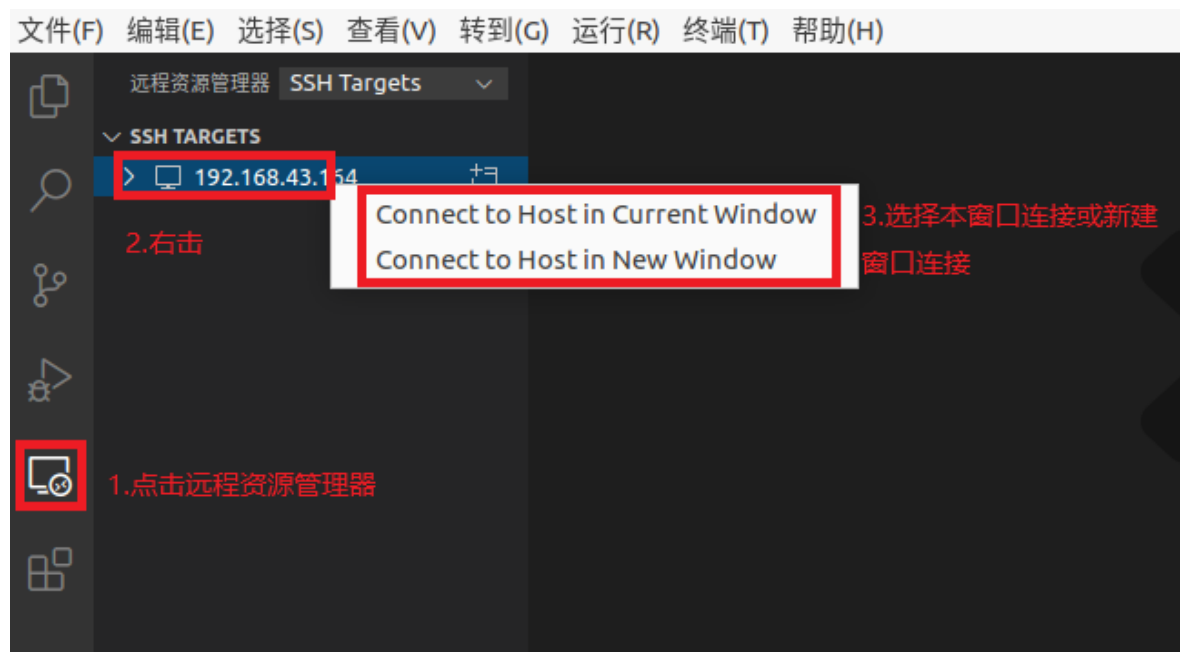


4.使用

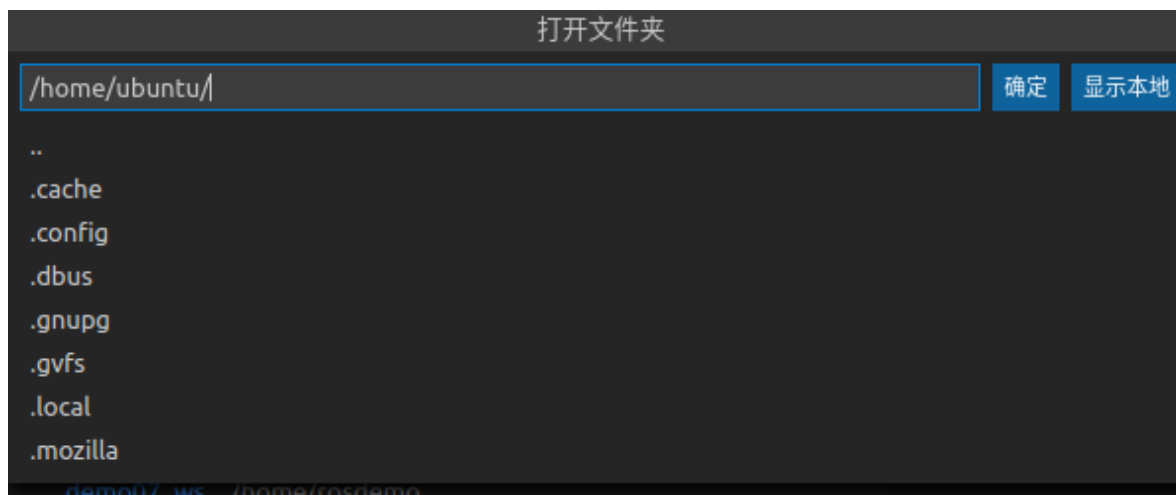
步骤1: 继续使用快捷键 `ctrl + shift + p` 打开命令输入窗口，并输入 `Remote-SSH:Connect to Host...`，此时列表中将显示步骤3中配置的ip地址，直接选择，选择后，VScode将打开一个新的窗口。



或者，也可以点击侧边栏的 `远程资源管理器`，在弹出的服务器列表中选择要连接的服务器，并右击，选择在本窗口或新窗口中实现远程连接。



步骤2: 选择菜单栏的 `文件` 下的 `打开文件夹`，在弹窗列表中选择需要打开的文件夹并点击确定即可。



最终，我们就可以像操作本地文件一样实现远程开发了。

9.3 导航实现

本节介绍实体机器人导航的基本实现流程。该流程实现与7.2节内容类似，主要内容仍然集中在SLAM、地图服务、定位与路径规划，本节内容不再重复介绍7.2节中各个知识点的实现细节，而是注重知识点应用。

实体机器人导航实现流程如下：

1. 准备工作；
2. SLAM实现；
3. 地图服务；
4. 定位实现；
5. 路径规划；

本节最后还会将导航与SLAM结合，实现自主移动的SLAM建图。

9.3.1 导航实现01_准备工作

1.1 分布式架构

分布式架构搭建完毕且能正常运行，在PC端可以远程登陆机器人端。

1.2 功能包安装

在机器人端安装导航所需功能包：

- 安装 gmapping 包(用于构建地图): `sudo apt install ros-<ROS版本>-gmapping`
- 安装地图服务包(用于保存与读取地图): `sudo apt install ros-<ROS版本>-map-server`
- 安装 navigation 包(用于定位以及路径规划): `sudo apt install ros-<ROS版本>-navigation`

新建功能（包名自定义，比如：nav），并导入依赖: gmapping map_server amcl move_base

1.3 机器人模型以及坐标变换

机器人的不同部件有不同的坐标系，我们需要将这些坐标系集成进同一坐标树，实现方案有两种：

1. 不同的部件相对于机器人底盘其位置都是固定的，可以通过发布静态坐标变换以实现集成；
2. 可以通过加载机器人URDF文件结合 `robot_state_publisher`、`joint_state_publisher`实现不同坐标系的集成。

方案1在上一章中已做演示，接下来介绍方案2的实现。

1.3.1 创建机器人模型相关的功能包

创建功能包: `catkin_create_pkg mycar_description urdf xacro`。

1.3.2 准备机器人模型文件

在功能包下新建 `urdf` 目录，编写具体的 `urdf` 文件（机器人模型相关URDF文件的编写可以参考第6章内容），示例如下：

文件`car.urdf.xacro`用于集成不同的机器人部件，内容如下：

```
1 <robot name="mycar" xmlns:xacro="http://wiki.ros.org/xacro">
2
3     <xacro:include filename="car_base.urdf.xacro" />
4     <xacro:include filename="car_camera.urdf.xacro" />
5     <xacro:include filename="car_laser.urdf.xacro" />
6
7 </robot>
```

文件`car_base.urdf.xacro`机器人底盘实现，内容如下：

```
1 <robot name="mycar" xmlns:xacro="http://wiki.ros.org/xacro">
2
3     <xacro:property name="footprint_radius" value="0.001" />
4     <link name="base_footprint">
5         <visual>
6             <geometry>
7                 <sphere radius="${footprint_radius}" />
8             </geometry>
9         </visual>
10    </link>
11
12
13    <xacro:property name="base_radius" value="0.1" />
14    <xacro:property name="base_length" value="0.08" />
15    <xacro:property name="lidi" value="0.015" />
16    <xacro:property name="base_joint_z" value="${base_length / 2 + lidi}" />
17    <link name="base_link">
18        <visual>
19            <geometry>
20                <cylinder radius="0.1" length="0.08" />
21            </geometry>
22
```

```

23         <origin xyz="0 0 0" rpy="0 0 0" />
24
25         <material name="baselink_color">
26             <color rgba="1.0 0.5 0.2 0.5" />
27         </material>
28     </visual>
29
30 </link>
31
32 <joint name="link2footprint" type="fixed">
33     <parent link="base_footprint" />
34     <child link="base_link" />
35     <origin xyz="0 0 0.055" rpy="0 0 0" />
36 </joint>
37
38
39
40 <xacro:property name="wheel_radius" value="0.0325" />
41 <xacro:property name="wheel_length" value="0.015" />
42 <xacro:property name="PI" value="3.1415927" />
43 <xacro:property name="wheel_joint_z" value="${(base_length / 2 + lidi -
wheel_radius) * -1}" />
44
45
46 <xacro:macro name="wheel_func" params="wheel_name flag">
47
48     <link name="${wheel_name}_wheel">
49         <visual>
50             <geometry>
51                 <cylinder radius="${wheel_radius}"
length="${wheel_length}" />
52             </geometry>
53
54             <origin xyz="0 0 0" rpy="${PI / 2} 0 0" />
55
56             <material name="wheel_color">
57                 <color rgba="0 0 0 0.3" />
58             </material>
59         </visual>
60
61     </link>
62
63     <joint name="${wheel_name}2link" type="continuous">
64         <parent link="base_link" />
65         <child link="${wheel_name}_wheel" />
66
67         <origin xyz="0 ${0.1 * flag} ${wheel_joint_z}" rpy="0 0 0" />
68         <axis xyz="0 1 0" />
69     </joint>
70
71 </xacro:macro>
72

```

```

73     <xacro:wheel_func wheel_name="left" flag="1" />
74     <xacro:wheel_func wheel_name="right" flag="-1" />
75
76
77
78     <xacro:property name="small_wheel_radius" value="0.0075" />
79     <xacro:property name="small_joint_z" value="${(base_length / 2 + lidi -
small_wheel_radius) * -1}" />
80
81     <xacro:macro name="small_wheel_func" params="small_wheel_name flag">
82         <link name="${small_wheel_name}_wheel">
83             <visual>
84                 <geometry>
85                     <sphere radius="${small_wheel_radius}" />
86                 </geometry>
87
88                 <origin xyz="0 0 0" rpy="0 0 0" />
89
90                 <material name="wheel_color">
91                     <color rgba="0 0 0 0.3" />
92                 </material>
93             </visual>
94
95         </link>
96
97         <joint name="${small_wheel_name}2link" type="continuous">
98             <parent link="base_link" />
99             <child link="${small_wheel_name}_wheel" />
100
101             <origin xyz="${0.08 * flag} 0 ${small_joint_z}" rpy="0 0 0" />
102             <axis xyz="0 1 0" />
103         </joint>
104
105     </xacro:macro >
106     <xacro:small_wheel_func small_wheel_name="front" flag="1"/>
107     <xacro:small_wheel_func small_wheel_name="back" flag="-1"/>
108
109 </robot>

```

文件car_camera.urdf.xacro机器人摄像头实现，内容如下：

```

1  <robot name="mycar" xmlns:xacro="http://wiki.ros.org/xacro">
2
3     <xacro:property name="camera_length" value="0.02" />
4     <xacro:property name="camera_width" value="0.05" />
5     <xacro:property name="camera_height" value="0.05" />
6     <xacro:property name="joint_camera_x" value="0.08" />
7     <xacro:property name="joint_camera_y" value="0" />
8     <xacro:property name="joint_camera_z" value="${base_length / 2 +
camera_height / 2}" />
9
10     <link name="camera">

```

```

11         <visual>
12             <geometry>
13                 <box size="${camera_length} ${camera_width} ${camera_height}"
14             />
15             </geometry>
16             <origin xyz="0 0 0" rpy="0 0 0" />
17             <material name="black">
18                 <color rgba="0 0 0 0.8" />
19             </material>
20         </visual>
21     </link>
22
23     <joint name="camera2base" type="fixed">
24         <parent link="base_link" />
25         <child link="camera" />
26         <origin xyz="${joint_camera_x} ${joint_camera_y} ${joint_camera_z}"
27         rpy="0 0 0" />
28     </joint>
29
30 </robot>

```

文件car_laser.urdf.xacro机器人雷达实现，内容如下：

```

1  <robot name="mycar" xmlns:xacro="http://wiki.ros.org/xacro">
2
3      <xacro:property name="support_radius" value="0.01" />
4      <xacro:property name="support_length" value="0.15" />
5
6      <xacro:property name="laser_radius" value="0.03" />
7      <xacro:property name="laser_length" value="0.05" />
8
9      <xacro:property name="joint_support_x" value="0" />
10     <xacro:property name="joint_support_y" value="0" />
11     <xacro:property name="joint_support_z" value="${base_length / 2 +
12     support_length / 2}" />
13
14     <xacro:property name="joint_laser_x" value="0" />
15     <xacro:property name="joint_laser_y" value="0" />
16     <xacro:property name="joint_laser_z" value="${support_length / 2 +
17     laser_length / 2}" />
18
19     <link name="support">
20         <visual>
21             <geometry>
22                 <cylinder radius="${support_radius}"
23                 length="${support_length}" />
24             </geometry>
25             <material name="yellow">
26                 <color rgba="0.8 0.5 0.0 0.5" />
27             </material>
28         </visual>
29     </link>

```



```

27     </link>
28
29     <joint name="support2base" type="fixed">
30         <parent link="base_link" />
31         <child link="support"/>
32         <origin xyz="${joint_support_x} ${joint_support_y}
33 ${joint_support_z}" rpy="0 0 0" />
34     </joint>
35     <link name="laser">
36         <visual>
37             <geometry>
38                 <cylinder radius="${laser_radius}" length="${laser_length}"
39 />
40             </geometry>
41             <material name="black">
42                 <color rgba="0 0 0 0.5" />
43             </material>
44         </visual>
45     </link>
46
47     <joint name="laser2support" type="fixed">
48         <parent link="support" />
49         <child link="laser"/>
50         <origin xyz="${joint_laser_x} ${joint_laser_y} ${joint_laser_z}"
51 rpy="0 0 0" />
52     </joint>
53 </robot>

```

1.3.3 在launch文件加载机器人模型

launch 文件（文件名称自定义，比如：car.launch）内容示例如下：

```

1 <launch>
2     <param name="robot_description" command="$(find xacro)/xacro $(find
3 mycar_description)/urdf/car.urdf.xacro" />
4     <node pkg="joint_state_publisher" name="joint_state_publisher"
5 type="joint_state_publisher" />
6     <node pkg="robot_state_publisher" name="robot_state_publisher"
7 type="robot_state_publisher" />
8 </launch>

```

为了使用方便，还可以将该文件包含进启动机器人的launch文件中，示例如下：

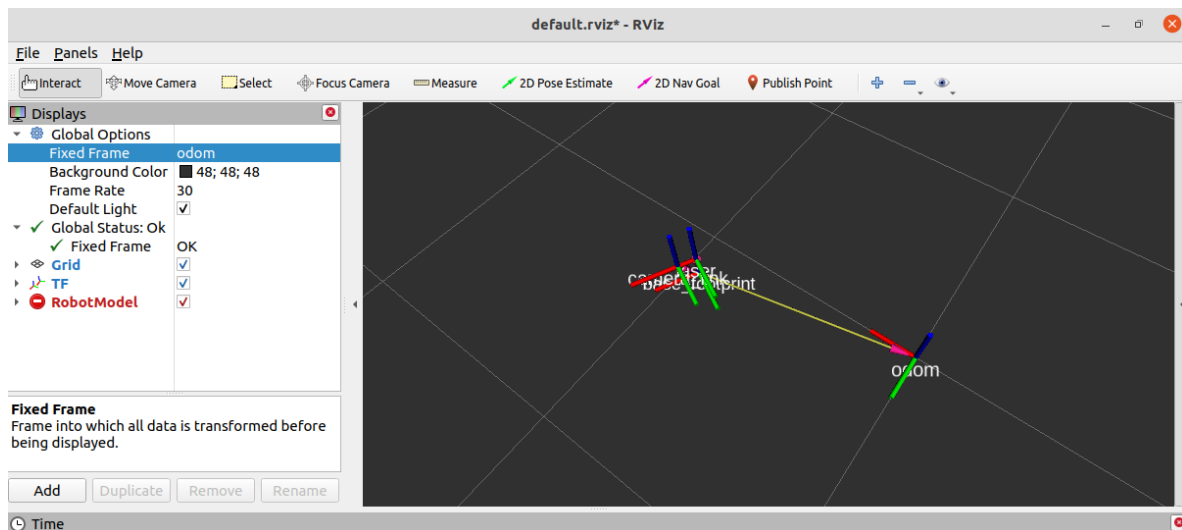
```

1 <launch>
2     <include file="$(find ros_arduino_python)/launch/arduino.launch" />
3     <include file="$(find usb_cam)/launch/usb_cam-test.launch" />
4     <include file="$(find rplidar_ros)/launch/rplidar.launch" />
5     <!-- 机器人模型加载文件 -->
6     <include file="$(find mycar_description)/launch/car.launch" />
7 </launch>

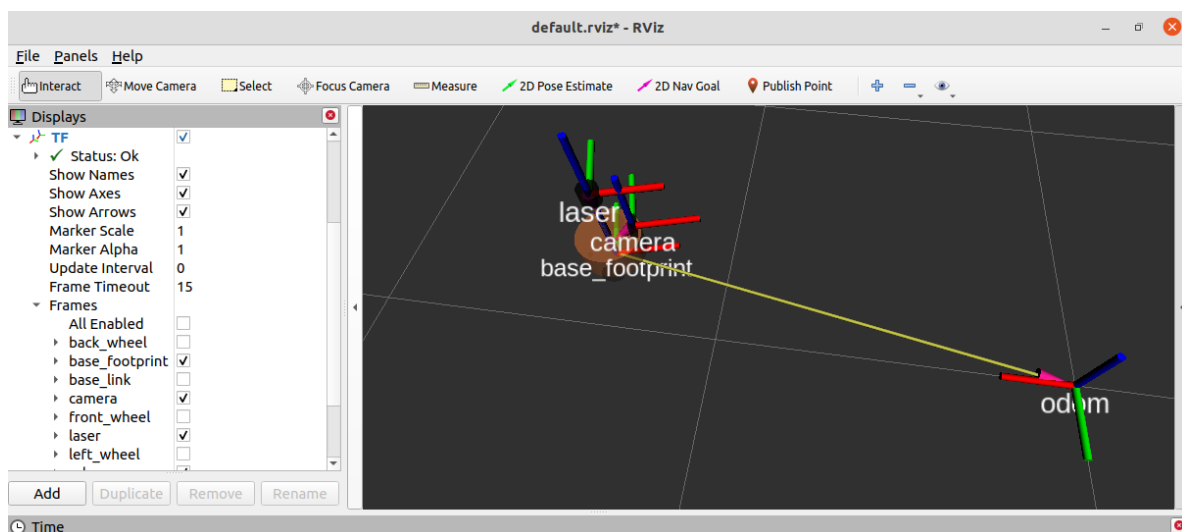
```

1.4结果演示

不使用机器人模型时，机器人端启动机器人(使用包含TF坐标换的launch文件)，从机端启动rviz，在rviz中添加RobotModel与TF组件，rviz中结果(此时显示机器人模型异常，且TF中只有代码中发布的坐标变换):



使用机器人模型时，机器人端加载机器人模型（执行上一步的launch文件）且启动机器人，从机端启动rviz，，在rviz中添加RobotModel与TF组件rviz中结果(此时显示机器人模型，且TF坐标变换正常):



后续，在导航时使用机器人模型。

9.3.2 导航实现02_SLAM建图

关于建图实现，仍然选用第7章中学习的：gmapping，实现如下：

2.1编写gmapping节点相关launch文件

在上一节创建的导航功能包中新建 launch 目录，并新建launch文件（文件名自定义，比如：gmapping.launch），代码示例如下：

```
1 <launch>
```

```

2      <node pkg="gmapping" type="slam_gmapping" name="slam_gmapping"
output="screen">
3          <remap from="scan" to="scan"/>
4          <param name="base_frame" value="base_footprint"/><!--底盘坐标系-->
5          <param name="odom_frame" value="odom"/> <!--里程计坐标系-->
6          <param name="map_update_interval" value="5.0"/>
7          <param name="maxUrange" value="16.0"/>
8          <param name="sigma" value="0.05"/>
9          <param name="kernelSize" value="1"/>
10         <param name="lstep" value="0.05"/>
11         <param name="astep" value="0.05"/>
12         <param name="iterations" value="5"/>
13         <param name="lsigma" value="0.075"/>
14         <param name="ogain" value="3.0"/>
15         <param name="lskip" value="0"/>
16         <param name="srr" value="0.1"/>
17         <param name="srt" value="0.2"/>
18         <param name="str" value="0.1"/>
19         <param name="stt" value="0.2"/>
20         <param name="linearUpdate" value="1.0"/>
21         <param name="angularUpdate" value="0.5"/>
22         <param name="temporalUpdate" value="3.0"/>
23         <param name="resampleThreshold" value="0.5"/>
24         <param name="particles" value="30"/>
25         <param name="xmin" value="-50.0"/>
26         <param name="ymin" value="-50.0"/>
27         <param name="xmax" value="50.0"/>
28         <param name="ymax" value="50.0"/>
29         <param name="delta" value="0.05"/>
30         <param name="llsamplerange" value="0.01"/>
31         <param name="llsamplestep" value="0.01"/>
32         <param name="lasamplerange" value="0.005"/>
33         <param name="lasamplestep" value="0.005"/>
34     </node>
35 </launch>

```

关键代码解释:

```

1  <remap from="scan" to="scan"/><!-- 雷达话题 -->
2  <param name="base_frame" value="base_footprint"/><!--底盘坐标系-->
3  <param name="odom_frame" value="odom"/> <!--里程计坐标系-->

```

2.2 执行

1. 执行相关launch文件，启动机器人并加载机器人模型：`roslaunch mycar_start start.launch`；

2. 启动地图绘制的 launch 文件：`roslaunch nav gmapping.launch`；

3. 启动键盘控制节点，用于控制机器人运动建图：`roslaunch teleop_twist_keyboard teleop_twist_keyboard.py`

4.在 rviz 中添加地图显示组件，通过键盘控制机器人运动，同时，在rviz中可以显示gmapping发布的栅格地图数据了，该显示结果与仿真环境下类似。下一步，还需要将地图单独保存。

9.3.3 导航实现03_地图服务

可以通过 map_server 实现地图的保存与读取。

3.1地图保存launch文件

首先在自定义的导航功能包下新建 map 目录，用于保存生成的地图数据。地图保存的语法比较简单，编写一个launch文件，内容如下：

```
1 <launch>
2   <arg name="filename" value="$(find nav)/map/nav" />
3   <node name="map_save" pkg="map_server" type="map_saver" args="-f $(arg
  filename)" />
4 </launch>
```

其中通过 filename 指定了地图的保存路径以及保存的文件名称。

SLAM建图完毕后，执行该launch文件即可。

测试：

首先，参考上一节，依次启动仿真环境，键盘控制节点与SLAM节点；

然后，通过键盘控制机器人运动并绘图；

最后，通过上述地图保存方式保存地图。

结果：在指定路径下会生成两个文件，xxx.pgm 与 xxx.yaml

3.2地图读取

通过 map_server 的 map_server 节点可以读取栅格地图数据，编写 launch 文件如下：

```
1 <launch>
2   <!-- 设置地图的配置文件 -->
3   <arg name="map" default="nav.yaml" />
4   <!-- 运行地图服务器，并且加载设置的地图-->
5   <node name="map_server" pkg="map_server" type="map_server" args="$(find
  mycar_nav)/map/$(arg map)"/>
6 </launch>
```

其中参数是地图描述文件的资源路径，执行该launch文件，该节点会发布话题:map(nav_msgs/OccupancyGrid)，最后，在 rviz 中使用 map 组件可以显示栅格地图。

9.3.4 导航实现04_定位

在ROS的导航功能包集navigation中提供了 amcl 功能包，用于实现导航中的机器人定位。

4.1编写amcl节点相关的launch文件

```
1 <launch>
2   <node pkg="amcl" type="amcl" name="amcl" output="screen">
3     <!-- Publish scans from best pose at a max of 10 Hz -->
4     <param name="odom_model_type" value="diff"/><!-- 里程计模式为差分 -->
5     <param name="odom_alpha5" value="0.1"/>
6     <param name="transform_tolerance" value="0.2" />
7     <param name="gui_publish_rate" value="10.0"/>
8     <param name="laser_max_beams" value="30"/>
9     <param name="min_particles" value="500"/>
10    <param name="max_particles" value="5000"/>
11    <param name="kld_err" value="0.05"/>
12    <param name="kld_z" value="0.99"/>
13    <param name="odom_alpha1" value="0.2"/>
14    <param name="odom_alpha2" value="0.2"/>
15    <!-- translation std dev, m -->
16    <param name="odom_alpha3" value="0.8"/>
17    <param name="odom_alpha4" value="0.2"/>
18    <param name="laser_z_hit" value="0.5"/>
19    <param name="laser_z_short" value="0.05"/>
20    <param name="laser_z_max" value="0.05"/>
21    <param name="laser_z_rand" value="0.5"/>
22    <param name="laser_sigma_hit" value="0.2"/>
23    <param name="laser_lambda_short" value="0.1"/>
24    <param name="laser_lambda_short" value="0.1"/>
25    <param name="laser_model_type" value="likelihood_field"/>
26    <!-- <param name="laser_model_type" value="beam"/> -->
27    <param name="laser_likelihood_max_dist" value="2.0"/>
28    <param name="update_min_d" value="0.2"/>
29    <param name="update_min_a" value="0.5"/>
30
31    <param name="odom_frame_id" value="odom"/><!-- 里程计坐标系 -->
32    <param name="base_frame_id" value="base_footprint"/><!-- 添加机器人基坐标
系 -->
33    <param name="global_frame_id" value="map"/><!-- 添加地图坐标系 -->
34
35  </node>
36 </launch>
```

4.2编写测试launch文件

amcl节点是不可以单独运行的，运行 amcl 节点之前，需要先加载全局地图，然后启动 rviz 显示定位结果，上述节点可以集成进launch文件，内容示例如下：

```

1 <launch>
2   <!-- 设置地图的配置文件 -->
3   <arg name="map" default="nav.yaml" />
4   <!-- 运行地图服务器，并且加载设置的地图-->
5   <node name="map_server" pkg="map_server" type="map_server" args="$(find
nav)/map/$(arg map)"/>
6   <!-- 启动AMCL节点 -->
7   <include file="$(find nav)/launch/amcl.launch" />
8
9 </launch>

```

当然，launch文件中地图服务节点和amcl节点中的包名、文件名需要根据自己的设置修改。

4.3 执行

1. 执行相关launch文件，启动机器人并加载机器人模型：`roslaunch mycar_start start.launch`；

2. 启动键盘控制节点：`roslaunch teleop_twist_keyboard teleop_twist_keyboard.py`；

3. 启动上一步中集成地图服务、amcl的launch文件：`roslaunch nav test_amcl.launch`；

4. 启动rviz并添加RobotModel、Map组件，分别显示机器人模型与地图，添加posearray插件，设置topic为particlecloud来显示amcl预估的当前机器人的位姿，箭头越是密集，说明当前机器人处于此位置的概率越高；

5. 通过键盘控制机器人运动，会发现posearray也随之而改变。运行结果与仿真环境下类似。

9.3.5 导航实现05_路径规划

路径规划仍然使用navigation功能包集中的move_base功能包。

5.1 编写launch文件

关于move_base节点的调用，模板如下：

```

1 <launch>
2
3   <node pkg="move_base" type="move_base" respawn="false" name="move_base"
output="screen" clear_params="true">
4     <rosparam file="$(find nav)/param/costmap_common_params.yaml"
command="load" ns="global_costmap" />
5     <rosparam file="$(find nav)/param/costmap_common_params.yaml"
command="load" ns="local_costmap" />
6     <rosparam file="$(find nav)/param/local_costmap_params.yaml"
command="load" />
7     <rosparam file="$(find nav)/param/global_costmap_params.yaml"
command="load" />
8     <rosparam file="$(find nav)/param/base_local_planner_params.yaml"
command="load" />
9   </node>
10
11 </launch>

```

5.2编写配置文件

可参考仿真实现。

1.costmap_common_params.yaml

该文件是move_base 在全局路径规划与本地路径规划时调用的通用参数，包括:机器人的尺寸、距离障碍物的安全距离、传感器信息等。配置参考如下：

```

1 #机器人几何参，如果机器人是圆形，设置 robot_radius,如果是其他形状设置 footprint
2 robot_radius: 0.12 #圆形
3 # footprint: [[-0.12, -0.12], [-0.12, 0.12], [0.12, 0.12], [0.12, -0.12]] #其
其他形状
4
5 obstacle_range: 3.0 # 用于障碍物探测，比如：值为 3.0，意味着检测到距离小于 3 米的
障碍物时，就会引入代价地图
6 raytrace_range: 3.5 # 用于清除障碍物，比如：值为 3.5，意味着清除代价地图中 3.5 米
以外的障碍物
7
8
9 #膨胀半径，扩展在碰撞区域以外的代价区域，使得机器人规划路径避开障碍物
10 inflation_radius: 0.2
11 #代价比例系数，越大则代价值越小
12 cost_scaling_factor: 3.0
13
14 #地图类型
15 map_type: costmap
16 #导航包所需要的传感器
17 observation_sources: scan
18 #对传感器的坐标系和数据配置。这个也会用于代价地图添加和清除障碍物。例如，你可以
用激光雷达传感器用于在代价地图添加障碍物，再添加kinect用于导航和清除障碍物。
19 scan: {sensor_frame: laser, data_type: LaserScan, topic: scan, marking: true,
clearing: true}

```

2.global_costmap_params.yaml

该文件用于全局代价地图参数设置:

```
1 global_costmap:
2   global_frame: map #地图坐标系
3   robot_base_frame: base_footprint #机器人坐标系
4   # 以此实现坐标变换
5
6   update_frequency: 1.0 #代价地图更新频率
7   publish_frequency: 1.0 #代价地图的发布频率
8   transform_tolerance: 0.5 #等待坐标变换发布信息的超时时间
9
10  static_map: true # 是否使用一个地图或者地图服务器来初始化全局代价地图, 如果不使用静态地图, 这个参数为false.
```

3.local_costmap_params.yaml

该文件用于局部代价地图参数设置:

```
1 local_costmap:
2   global_frame: odom #里程计坐标系
3   robot_base_frame: base_footprint #机器人坐标系
4
5   update_frequency: 10.0 #代价地图更新频率
6   publish_frequency: 10.0 #代价地图的发布频率
7   transform_tolerance: 0.5 #等待坐标变换发布信息的超时时间
8
9   static_map: false #不需要静态地图, 可以提升导航效果
10  rolling_window: true #是否使用动态窗口, 默认为false, 在静态的全局地图中, 地图不会变化
11  width: 3 # 局部地图宽度 单位是 m
12  height: 3 # 局部地图高度 单位是 m
13  resolution: 0.05 # 局部地图分辨率 单位是 m, 一般与静态地图分辨率保持一致
```

4.base_local_planner_params

基本的局部规划器参数配置, 这个配置文件设定了机器人的最大和最小速度限制值, 也设定了加速度的阈值。

```
1 TrajectoryPlannerROS:
2
3 # Robot Configuration Parameters
4 max_vel_x: 0.5 # X 方向最大速度
5 min_vel_x: 0.1 # X 方向最小速度
6
7 max_vel_theta: 1.0 #
8 min_vel_theta: -1.0
9 min_in_place_vel_theta: 1.0
10
11 acc_lim_x: 1.0 # X 加速限制
12 acc_lim_y: 0.0 # Y 加速限制
```



```

13   acc_lim_theta: 0.6 # 角速度加速限制
14
15   # Goal Tolerance Parameters, 目标公差
16   xy_goal_tolerance: 0.10
17   yaw_goal_tolerance: 0.05
18
19   # Differential-drive robot configuration
20   # 是否是全向移动机器人
21   holonomic_robot: false
22
23   # Forward Simulation Parameters, 前进模拟参数
24   sim_time: 0.8
25   vx_samples: 18
26   vtheta_samples: 20
27   sim_granularity: 0.05

```

5.3 launch 文件集成

如果要实现导航，需要集成地图服务、amcl、move_base 等，集成示例如下：

```

1   <launch>
2     <!-- 设置地图的配置文件 -->
3     <arg name="map" default="nav.yaml" />
4     <!-- 运行地图服务器，并且加载设置的地图-->
5     <node name="map_server" pkg="map_server" type="map_server" args="$(find
nav)/map/$(arg map)" />
6     <!-- 启动AMCL节点 -->
7     <include file="$(find nav)/launch/amcl.launch" />
8
9     <!-- 运行move_base节点 -->
10    <include file="$(find nav)/launch/move_base.launch" />
11
12  </launch>

```

5.4 测试

1. 执行相关launch文件，启动机器人并加载机器人模型：`roslaunch mycar_start start.launch`；

2. 启动导航相关的 launch 文件：`roslaunch nav nav.launch`；

3. 添加Rviz组件实现导航（参考仿真实现）。

9.3.6 导航与SLAM建图

与仿真环境类似的，也可以实现机器人自主移动的SLAM建图，步骤如下：

1. 编写launch文件，集成SLAM与move_base相关节点；
2. 执行launch文件并测试。

6.1编写launch文件

当前launch文件（名称自定义，比如：auto_slam.launch）实现，无需调用map_server的相关节点，只需要启动SLAM节点与move_base节点，示例内容如下：

```
1 <launch>
2   <!-- 启动SLAM节点 -->
3   <include file="$(find nav)/launch/gmapping.launch" />
4   <!-- 运行move_base节点 -->
5   <include file="$(find nav)/launch/move_base.launch" />
6 </launch>
```

6.2测试

- 1.执行相关launch文件，启动机器人并加载机器人模型：`roslaunch mycar_start start.launch`；
- 2.然后执行当前launch文件：`roslaunch nav auto_slam.launch`；
- 3.在rviz中通过2D Nav Goal设置目标点，机器人开始自主移动并建图了；
- 4.最后可以使用 map_server 保存地图。

9.4 本章小结

本章主要介绍关于实体机器人导航实现的，主要内容如下：

- 仿真环境与真实环境下的区别；
- VSCode远程开发环境的搭建；
- 实体机器人的导航实现。

整体而言，将仿真环境下的导航功能迁移到实体机器人并不复杂，不过在实际迁移时，需要调整导航相关的一些参数。

